

Understanding LangChain Document Loaders

Page 1 — What Are Document Loaders?

Introduction

LangChain is a framework for building applications with Large Language Models (LLMs). It provides tools for connecting LLMs with external information such as PDFs, websites, databases, and text files.

An LLM cannot automatically access your private documents. **Document Loaders** solve this problem by reading data from different sources and converting it into a format that LangChain can process.

What Is a Document Loader?

A document loader reads information from a specific source and creates LangChain **Document** objects.

Common sources include:

- PDF files
- Text files
- CSV files
- Websites
- Word documents
- Databases
- Online services

A document typically contains:

```
Document(  
  
    page_content="LangChain is a framework...",  
  
    metadata={"source": "guide.pdf", "page": 1}  
  
)
```

page_content contains the extracted text, while **metadata** stores information such as the filename, page number, or URL.

Why Are Loaders Important?

Document loaders are usually the first step in a RAG application:

Data → Loader → Documents → Splitter → Embeddings → Vector Store → LLM

Think of a document loader as a **bridge between raw data and your AI application**.

Page 2 — Types of Document Loaders

LangChain provides different loaders depending on the type of data you want to process.

PDF Loader

PDF loaders extract text from PDF documents.

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("document.pdf")
```

```
documents = loader.load()
```

PDF loaders are useful for research papers, manuals, reports, books, and company documentation.

Text Loader

For `.txt` files:

```
from langchain_community.document_loaders import TextLoader
```

```
loader = TextLoader("notes.txt")
```

```
documents = loader.load()
```

CSV Loader

CSV loaders are useful for structured data such as product catalogs, FAQs, and datasets.

Web Loaders

Web loaders retrieve content from websites and convert it into documents. They are useful for building applications that answer questions from online documentation.

Directory Loaders

When you have many files, directory-based loading can process a collection of documents rather than loading every file individually.

Example:

```
knowledge/
```

```
├── guide.pdf
```

```
├── faq.pdf
```

└─ notes.txt

└─ policy.txt

Choosing a Loader

Source	Loader
PDF	PDF Loader
TXT	Text Loader
CSV	CSV Loader
Website	Web Loader
Multiple files	Directory Loader

Choose the loader based on your **data format and source**.

Page 3 — Document Loaders in RAG

What Is RAG?

Retrieval-Augmented Generation (RAG) allows an LLM to answer questions using external knowledge.

For example, a chatbot can search company PDFs to answer:

“What is the company's leave policy?”

Instead of relying only on the model's existing knowledge, the application retrieves relevant information from your documents.

Step 1: Load

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("policy.pdf")
```

```
documents = loader.load()
```

Step 2: Split

Large documents are divided into smaller chunks.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=1000,
```

```
    chunk_overlap=200
```

```
)
```

```
chunks = splitter.split_documents(documents)
```

Step 3: Embed and Store

The chunks are converted into embeddings and stored in a vector database.

Step 4: Retrieve and Generate

When a user asks a question, the retriever finds relevant chunks. These chunks are provided to the LLM as context, allowing it to generate a more relevant answer.

Key Concepts

Document Loader: Reads external data.

Document: Stores text and metadata.

Text Splitter: Breaks documents into chunks.

Embedding: Converts text into numerical vectors.

Vector Store: Stores embeddings for search.

Retriever: Finds relevant information.

Final Takeaway

Document loaders are a fundamental part of LangChain and RAG applications. They transform raw files and external data into documents that can be processed by the rest of the AI pipeline.

Remember:

Load → Split → Embed → Store → Retrieve → Generate

